

OFFLINE ANCHOR DAY · 5 HOURS

Session 17

Observability Deep Dive

Distributed Tracing · Metrics · Structured Logging
Micrometer + Zipkin + Prometheus + Grafana



Dr. ElSayed Mohamed Elsayed Baladoh

Tech Lead · Ph.D. · sayedbaladoh@yahoo.com

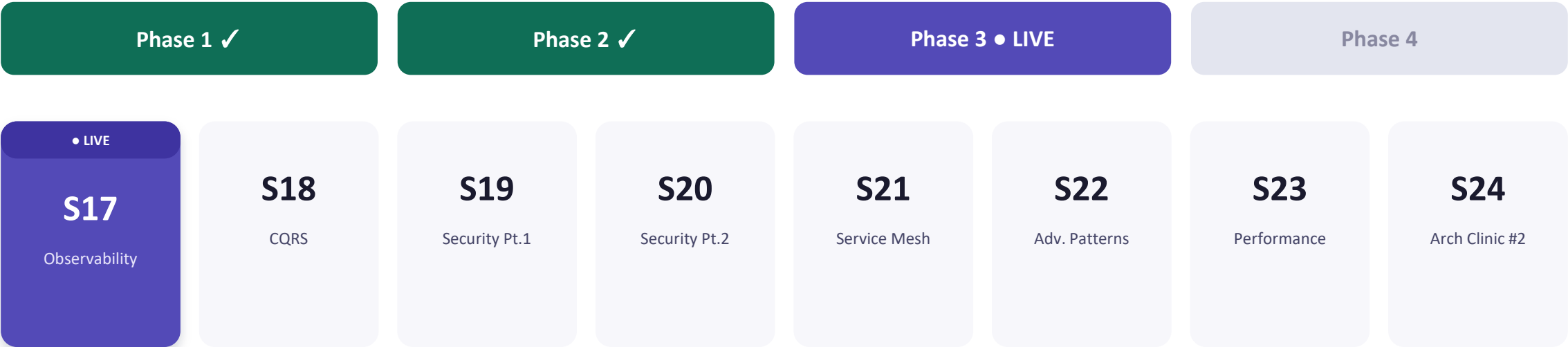


🕒 Wednesday · 10:00–3:00

🏗️ Phase 3 — Advanced & Enterprise

🚩 Session 17 of 29

Phase 3 — Advanced & Enterprise



Session 17 is the third Offline Anchor Day (after Sessions 1 and 9) — a full observability stack setup benefits from in-person troubleshooting and whiteboard time.

Note: the full 29-session roadmap lives in the Kickoff deck only.

Zipkin Has Been Running Since Session 1.

Since Session 1, **Zipkin** has been sitting in your docker-compose.yml, running quietly, doing nothing for you.

Today is the day we finally use it.

🔍 Why did it wait 17 sessions?

- OBG-001 (Session 1): infrastructure must not be introduced before its teaching session — even if it is technically running.
- There was nothing worth tracing until real cross-service complexity existed: no Gateway (S2), no resilience (S4-5), no Feign (S6), no Saga (S7).
- Today, for the first time, the platform has enough real complexity to make tracing worth teaching.

Session 16 Lab Review

- Helm chart install verified?
- ServiceAccount / Role present?
- session-16 committed before today?

Where Did the Request Break — Across 8 Services?

1

"My order took 9 seconds to confirm — and I'm not sure it even worked."

2

Order Service logs — grep by a rough timestamp. Nothing obvious, timestamp fuzzy.

3

Gateway logs — separate terminal, separate timestamp format, no shared request ID.

4

Inventory Service logs — a third terminal. Manually correlating by eyeballing timestamps.

5

Payment Service logs — a fourth terminal. 20 minutes in — still guessing which service caused it.

ROOT CAUSE

Resilience Made the System More Robust — and More Opaque



Every retry, fallback, and async Saga step (Sessions 4–7, 12) hides the true request path from a flat log file.

Ask the class:

"If every service printed its own timestamp and its own request ID, would that solve the problem?" — Not fully: request IDs must be the SAME id, generated once and propagated, not invented independently by each service.

Monitoring vs Observability



Monitoring

A question you already knew to ask

"Is Order Service CPU above 80%?"
"Is /actuator/health returning DOWN?"

Dashboards + alerts on KNOWN failure modes. What Session 15-16 probes already give you.



Observability

A question you did NOT know you'd need to ask

"Why did THIS SPECIFIC order, for THIS customer, take 9 seconds, when the average is 200ms?"

Requires deep, request-level, cross-service visibility — not a pre-built dashboard.

You cannot buy Observability as a single dashboard. You build it in through three complementary data types — the Three Pillars.

Traces · Metrics · Logs



TRACES

Where in the request path did time go?
Which service, which call?

Micrometer Tracing + Zipkin



METRICS

How is the system behaving in aggregate,
over time?

Micrometer + Prometheus + Grafana



LOGS

What exactly happened, in detail, at one point
in the code?

Logback (JSON) + Trace ID correlation

Metrics tell you SOMETHING is wrong. Traces tell you WHERE. Logs tell you WHY. None of the three replaces the others.

- *Metrics tell you SOMETHING is wrong (P99 latency spiked).*
- *Traces tell you WHERE (the Feign call to Inventory).*
- *Logs tell you WHY (a specific exception, with full context).*

What Is a Trace? What Is a Span?

TRACE

The complete journey of ONE request — from the Gateway to the final response (or a Saga's final state).
Identified by a single **Trace ID** shared by every span.

SPAN

ONE unit of work within that journey — e.g.
"OrderService.createOrder()", "Feign call to INVENTORY-SERVICE".
Each span has its own **Span ID** and a **reference** to its PARENT span.

```
traceId: 7a3f9c... (same for every span below)
├─ span: Gateway Path=/api/orders [0ms → 15ms]
|   └─ span: OrderService.createOrderAsync [15ms → 210ms]
|       └─ span: Feign → INVENTORY-SERVICE [20ms → 65ms]
|           └─ span: KafkaTemplate.send(order-events) [66ms → 70ms]
└─ span: InventorySagaHandler.handleOrderPlaced [72ms → 205ms] (separate thread)
```

One Trace, Five Spans — What a Real Order Looks Like

🔗 The last span belongs to the SAME trace even though it runs in a different service, on a different thread — this is why Kafka header propagation matters.

How Context Propagates

Communication Type	How Trace Context Travels	Session Reference
Gateway → Order Service	HTTP header: traceparent (W3C format)	Session 2-3 (Gateway)
Order → Inventory (Feign)	HTTP header, auto-propagated by Micrometer Tracing	Session 6 (OpenFeign)
Order → Kafka (Saga events)	Kafka message headers, auto-propagated	Session 7 (Choreography), 12 (Orchestration)

 *Production note: **Micrometer Tracing** auto-instruments Feign, RestTemplate, WebClient, and Spring Kafka out of the box once the dependencies are on the classpath — you rarely write propagation code by hand.*

Sampling Rate — How Much Tracing Data to Keep

Local development / this course?



probability: 1.0 — trace 100% of requests, see everything

Low-traffic production (<100 req/s)?



probability: 1.0 is often still affordable

High-traffic production (thousands req/s)?



probability: 0.05–0.10 — trace 5-10%

Need to guarantee every ERROR is traced?



Error-based sampling override (always sample on exception)

Zipkin storage/network becoming a bottleneck?



Lower the sampling probability, not the trace detail per request

Add Tracing Dependencies

`pom.xml` — add to `order-service`, `product-service`, `inventory-service`, `payment-service`, `api-gateway`

```
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-tracing-bridge-brave</artifactId>
</dependency>
<dependency>
  <groupId>io.zipkin.reporter2</groupId>
  <artifactId>zipkin-reporter-brave</artifactId>
</dependency>

<!-- spring-boot-starter-actuator must already be present (since Session 1) -->
```

Almost entirely dependency + configuration, not custom code.

Micrometer Tracing auto-instruments Feign, RestTemplate, WebClient, and Spring Kafka once the bridge dependency above is present.

Configure Sampling + Zipkin Endpoint

`application.yml` — add to every service in the request path

```
management:
  tracing:
    sampling:
      probability: 1.0          # DEV ONLY — see Engineering Decision above
  zipkin:
    tracing:
      endpoint: http://localhost:9411/api/v2/spans
spring:
  application:
    name: order-service        # shown as the service name in Zipkin UI
                                # ALREADY SET since Session 1 — nothing new here
```

Verify Zipkin is reachable

```
docker compose ps | grep zipkin  # zipkin has been running since Session 1
# zipkin    running    0.0.0.0:9411->9411/tcp
open http://localhost:9411       # Zipkin UI loads — currently empty
# Zipkin UI loads — currently empty, no traces recorded yet
```

One Request → One Connected Trace

```
curl -X POST http://localhost:8080/api/orders \
  -H 'Content-Type: application/json' \
  -H 'Authorization: Bearer <test-jwt>' \
  -d '{"productId":"PROD-001","quantity":1,"amount":100.00}'

# In Zipkin UI: Run Query -> serviceName: order-service
# Expected: one trace, ~5 spans, Gateway -> Order -> Feign -> Inventory
```



Show live: click into the trace — the waterfall view.

"This is the Feign call to Inventory — 45ms. This is the whole request — 210ms. Where would YOU look first if this were 9 seconds instead of 210ms?"

Async Saga Steps Share the Same Trace

```
pom.xml — add to order-service, inventory-service, payment-service (Kafka Saga)
```

```
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-tracing-bridge-brave</artifactId>
</dependency>

<!-- spring-kafka already on the classpath since Session 7 — Micrometer -->
<!-- Tracing auto-instruments KafkaTemplate.send() and @KafkaListener -->
<!-- once the bridge dependency above is present. -->
```

Demo: place an order triggering the S7 Choreography Saga (or S12 Orchestration).

In Zipkin, search by traceId — the Kafka consumer spans in Inventory Service and Payment Service appear as children of the SAME trace as the original HTTP request, even though they ran on a separate thread, separate process, milliseconds later.

Micrometer — "SLF4J for Metrics"

Just as **SLF4J** lets you write logging code once and swap Logback/Log4j2 underneath, **Micrometer** lets you write metrics code once and swap Prometheus/Datadog/CloudWatch underneath — without touching business code.

Spring Boot Actuator (used since Session 1 for /actuator/health) already includes Micrometer — every service has been silently recording basic metrics since Session 1.

🔗 Remember /actuator/circuitbreakers (Session 4)? /actuator/bulkheads (Session 5)?

Those Actuator endpoints are ALSO powered by Micrometer under the hood. Today we expose the raw metrics feed those endpoints are built on top of.

Counter · Gauge · Timer

Metric Type	Answers	Platform Example
Counter	How many times did X happen? (monotonically increasing)	Total orders created, total payment failures
Gauge	What is the current value of X right now?	Bulkhead available-concurrent-calls (Session 5)
Timer	How long did X take, and what's the distribution?	createOrder() latency, P50/P95/P99



A **Gauge** fluctuates *up and down*.
A **Counter** only *increases*.
A **Timer** captures **duration + distribution** — the same shape as P95/P99 latency you'll see in Session 23's load tests.

Prometheus PULLs. Zipkin is PUSHed to.



Prometheus

PULL-based

A time-series database that PULLS (scrapes) metrics from each service's /actuator/prometheus endpoint on a schedule (e.g. every 15 seconds).



Zipkin

PUSH-based

Services actively PUSH spans to the Zipkin server as they are generated — the opposite direction from Prometheus.

Grafana is a visualization layer on top of Prometheus (or any data source) — dashboards, graphs, alerts. Grafana **does not** collect metrics itself; it queries Prometheus.

Neither model is "better" — they are simply how each tool was designed.

Custom Metric + Prometheus Endpoint

`pom.xml` — Add Prometheus dependency

```
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-registry-prometheus</artifactId>
</dependency>
```

`application.yml` — Expose the endpoint

```
# application.yml
management:
  endpoints:
    web:
      exposure:
        include: health,info,prometheus,circuitbreakers,bulkheads # add prometheus to the existing list
```

Custom Metric + Prometheus + Grafana Endpoints

`OrderController.java` — Apply `@Timed` to a business-critical endpoint

```
@Timed(value = "order.create.duration", description = "Time to create an order")
@PostMapping
public CompletableFuture<ResponseEntity<OrderResponse>> createOrder(@RequestBody OrderRequest request) {
    return orderService.createOrderAsync(request).thenApply(ResponseEntity::ok);
}

// @Timed requires a TimedAspect bean — add once in a @Configuration class:
@Bean
public TimedAspect timedAspect(MeterRegistry registry) {
    return new TimedAspect(registry);
}
```

Scrape Config + Docker Compose

`prometheus.yml` — new file, mounted into the Prometheus container

```
# prometheus.yml
# Configure Prometheus to scrape our services
scrape_configs:
  - job_name: 'order-service'
    metrics_path: '/actuator/prometheus'
    static_configs:
      - targets: ['order-service:8082']
  - job_name: 'product-service'
    metrics_path: '/actuator/prometheus'
    static_configs:
      - targets: ['product-service:8081']
```

```
# docker-compose.yml
# add Prometheus + Grafana containers
prometheus:
  image: prom/prometheus:latest
  volumes:
    ["/prometheus.yml:/etc/prometheus/prometheus.yml"]
  ports: ["9090:9090"]
  networks: [platform-net]
grafana:
  image: grafana/grafana:latest
  ports: ["3000:3000"]
  networks: [platform-net]
```

Import a Grafana dashboard and observe

Grafana: Add data source -> Prometheus -> `http://prometheus:9090`
Dashboards -> Import -> ID 12900 (Spring Boot Statistics, community dashboard)

Grafana Populating Under Generated Load

```
for i in {1..50}; do curl -s -X POST http://localhost:8080/api/orders \
  -H 'Content-Type: application/json' \
  -d '{"productId":"PROD-001","quantity":1,"amount":10}'; done

# Watch order.create.duration and http.server.requests populate
# on the Grafana dashboard in near real-time
```



What this proves

Every service has been recording metrics silently since Session 1. Today we simply exposed and visualized what was already there — no new instrumentation logic, just a new lens on existing data.

Structured (JSON) vs Plain-Text Logging

High-traffic production service?



Structured (JSON) — mandatory

Small team, human reads logs directly on one machine?



Plain text is often OK

Centralized log aggregation (ELK / Loki) planned?



Structured — mandatory prerequisite

Multiple services, cross-service queries needed?



Structured — mandatory (grep by traceId)

Our platform: since we have 8+ services and Session 17's cross-service tracing, structured logging with traceId/spanId correlation is the correct default.

Logback JSON Encoder + Trace ID Correlation

`pom.xml`

```
<dependency>
  <groupId>net.logstash.logback</groupId>
  <artifactId>logstash-logback-encoder</artifactId>
  <version>7.4</version>
</dependency>
```

`logback-spring.xml`

```
<configuration>
  <appender name="JSON" class="ch.qos.logback.core.ConsoleAppender">
    <encoder class="net.logstash.logback.encoder.LogstashEncoder">
      <includeMdcKeyName>traceId</includeMdcKeyName>
      <includeMdcKeyName>spanId</includeMdcKeyName>
    </encoder>
  </appender>
  <root level="INFO"><appender-ref ref="JSON"/></root>
</configuration>
<!-- Micrometer Tracing automatically populates MDC with traceId/spanId — no extra code needed -->
```

Today: JSON Logs. NOT Today: The ELK Stack.

```
// Example resulting log line (one JSON object per line):  
{ "@timestamp": "2026-08-27T10:15:02.123Z", "level": "INFO",  
  "logger_name": "c.m.p.order.OrderService",  
  "message": "[SAGA] Order ORD-123 CONFIRMED",  
  "traceId": "7a3f9c2b1e8d4f6a", "spanId": "91ab3c" }  
  
# Now: grep '"traceId": "7a3f9c2b1e8d4f6a"' across ALL service log files  
# returns every log line from every service for THIS SPECIFIC request.
```

"We stop at structured JSON logs printed to console/file today. Centralizing them into a searchable store (Elasticsearch, Logstash, Kibana) is a production concern with real infrastructure cost — beyond this course's scope."

What you're building today is the PREREQUISITE for ELK, not ELK itself.

Tracing, Metrics & Structured Logging

product-service (primary) + order-service (bonus, includes Saga trace verification) · 50 min

1

Distributed Tracing

Add tracing deps + config to product-service; verify a trace appears in Zipkin UI

2

Custom Metric + Prometheus

@Timed on a business-critical endpoint; scrape via prometheus.yml

3

Structured JSON Logging

Logback JSON console appender; verify traceId matches the Zipkin trace

4

Verification

Existing Session 10 unit tests (@WebMvcTest, TestContainers) must still pass

What We Added Today

 Capability Added

End-to-End Observability. The platform can now answer "where did this specific request spend its time, and why?" across every service, sync and async.

 Services Updated

product-service + order-service: Micrometer Tracing + Zipkin, @Timed metric, structured JSON logs. Infra: Prometheus + Grafana added to docker-compose.yml.

Platform Progress — 8-Segment Tracker





Daily Quiz

8 Questions · 10 Minutes · Google Forms or Kahoot

Traces · Metrics · Structured Logging · Sampling Rate · Pull vs Push

5 Things That Will Go Wrong Today

No traces appear in Zipkin UI

Use `http://zipkin:9411/api/v2/spans` (Docker hostname) from inside a container, not localhost.

Trace missing spans from a downstream service

That service is missing `micrometer-tracing-bridge-brave` or `sampling.probability` is 0.

@Timed metric never appears in /actuator/prometheus

Missing the `TimedAspect @Bean` — same AOP requirement as Resilience4j (Session 4).

Prometheus target shows DOWN

Use the Docker Compose service name (`product-service:8081`), not localhost.

Kafka Saga spans don't share the HTTP request's traceId

Both producer AND consumer service need the tracing bridge on the classpath.



Session 17 Complete

Observability Layer Active — Traces · Metrics · Structured Logs

Next: Session 18 — CQRS Pattern
Monday, 3:00–5:30 PM, Online